

지구 반대편을 대응하는 어플리케이션을 위해

0년차 프론트엔드의 글로벌 프로젝트 삼질기

김민규 @cobocho

INTERNATIONALIZATION

프론트엔드에서의 국제화 요구사항



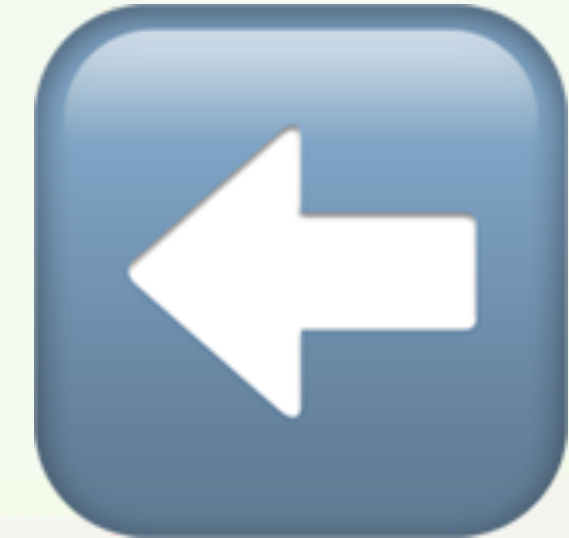
다국어 지원



통화



타임존



RTL

지구 반대편을 대응하는 어플리케이션을 위해

프론트엔드에서의 국제화 요구사항

Jessica Lau's N...

Search

Updates

Settings & members

New page

Teamspaces

Team Content

Teamspace Home

Employee Directory

Blog Calendar

Toothbrush HQ

The Floss Hub

Teamspace Home

Shared

Travel plans

Northern Ireland ro...

Ireland road trip

Japan trip

Colorado trip

Private

Getting Started

Quick Note

Personal Home

Task List

Team Content / Blog Calendar

Share ⌚ ☆ ⋮

Blog Calendar

All items Calendar view By content type By status By writer 1 more... Filter Sort ⚡ 🔍 ⋮ New

Aa Name	Status	Type of content	Writer	Reviewer	Publish date
10 Google Docs formatting features	Drafting	Blog	Jessica Lau	Nick Lucchesi	February 9, 2024
AI roundup #4	Done	Newsletter	Jessica Lau	Nick Lucchesi	February 6, 2024
The best marketing podcasts	Reviewi...	Blog	Jessica Lau	Nick Lucchesi	February 14, 2024
Podcast announcement	Backlog	Podcast	Jessica Lau	Nick Lucchesi	

+ New

Calculate

?

지구 반대편을 대응하는 어플리케이션을 위해

어떻게 구현할 것인가



같은 기능을 구현하면 같은 어플리케이션일까요?

지역 규칙의 등장

세금 계산

법적 요구사항

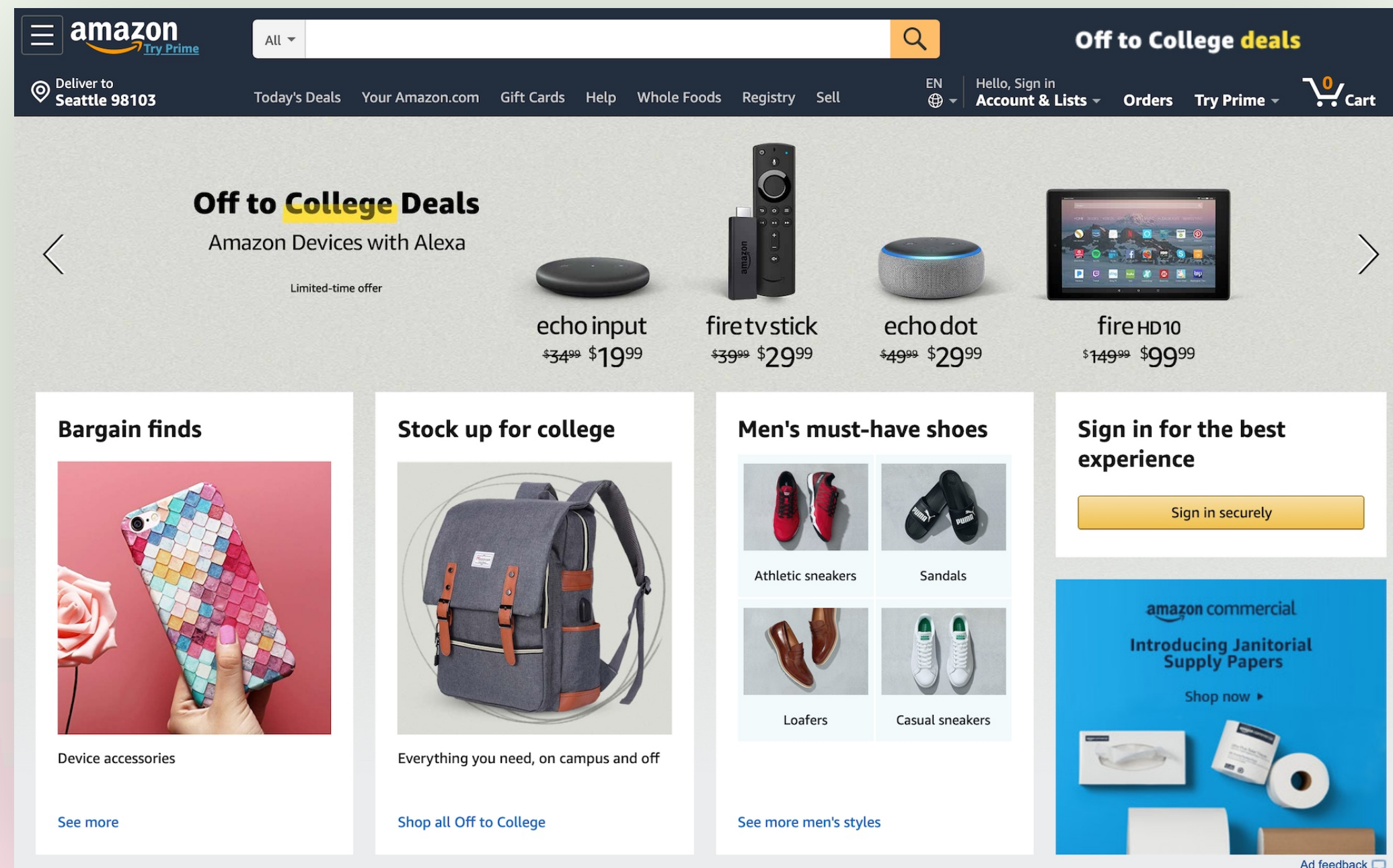
결제 수단

기관 연동

각 국가마다 적용되는 지역 특수적인 규칙

지구 반대편을 대응하는 어플리케이션을 위해

지역 규칙의 등장



같은 기능을 구현하지만 서로 다른 구현 세부사항

중복과 중복이 아닌것에 대한 경계

국제 프로토콜

게시판 관리 기능

규격 기기 관리

펌웨어 매니징

국가별로 중복되는 기능

공공기관 연동

주소 체계

정산 및 통계

멤버십 관리

국가별로 분리되는 기능

추상화에 대한 딜레마



쪼개기엔 중복되는게 많고... 합치기엔 분기가 많고...

지구 반대편을 대응하는 어플리케이션을 위해

모노레포를 통한 분리

TURBOREPO

패키지로 같이 사용되는 모듈을 분리하자

@repo/service

@repo/eslint

@repo/store

@repo/ui

@repo/utils

@repo/hooks

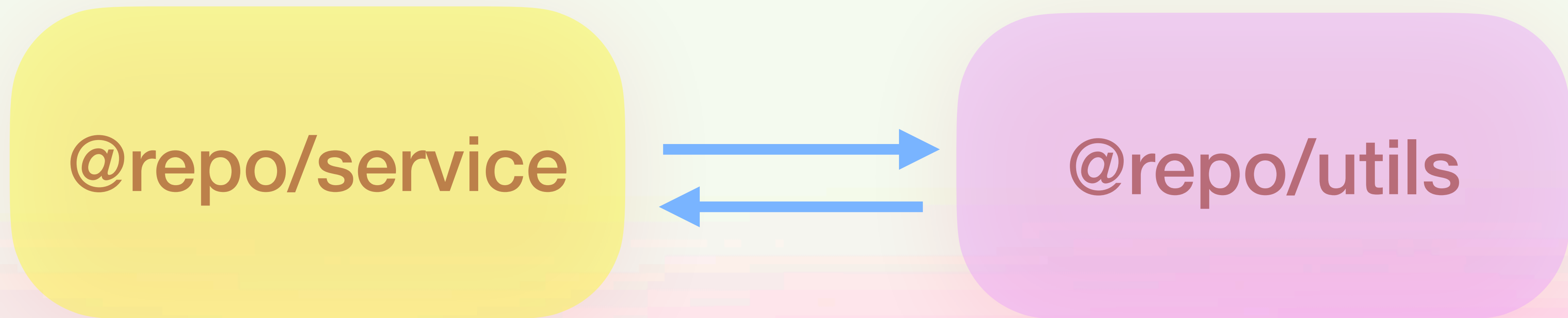
지구 반대편을 대응하는 어플리케이션을 위해

관리하기 복잡한 패키지 구조



패키지 관리 3시간 개발 10분

상호 참조 문제



모노레포 레이어 아키텍처

모노레포 레이어 아키텍처 원칙

하위 레이어는 상위 레이어를 의존하지 않는다

각각의 레이어들을 책임을 기준으로 분리한다

책임 이외에도 외부 모듈의 의존 또한 판단한다

지구 반대편을 대응하는 어플리케이션을 위해

모노레포 레이어 아키텍처

Upper Layer

Application

Lower Layer

지구 반대편을 대응하는 어플리케이션을 위해

모노레포 레이어 아키텍처

Upper Layer

Application

Application Layer

Packages

Lower Layer

지구 반대편을 대응하는 어플리케이션을 위해

모노레포 레이어 아키텍처

Upper Layer

app-kr

app-us

Application Layer

Packages

Lower Layer

모노레포 레이어 아키텍처

Upper Layer

app-kr

app-us

Application Layer

ev-core

Domain Layer

Packages

Lower Layer

모노레포 레이어 아키텍처

Upper Layer

app-kr

app-us

Application Layer

ev-core

Domain Layer

query

Service Layer

Packages

Lower Layer

모노레포 레이어 아키텍처

Upper Layer

app-kr

app-us

Application Layer

ev-core

Domain Layer

query

Service Layer

ui

UI Layer

Packages

Lower Layer

모노레포 레이어 아키텍처

Upper Layer

app-kr

app-us

Application Layer

ev-core

Domain Layer

query

Service Layer

ui

UI Layer

hooks

store

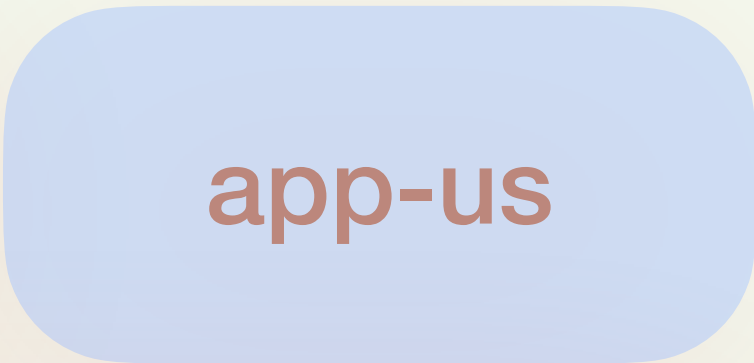
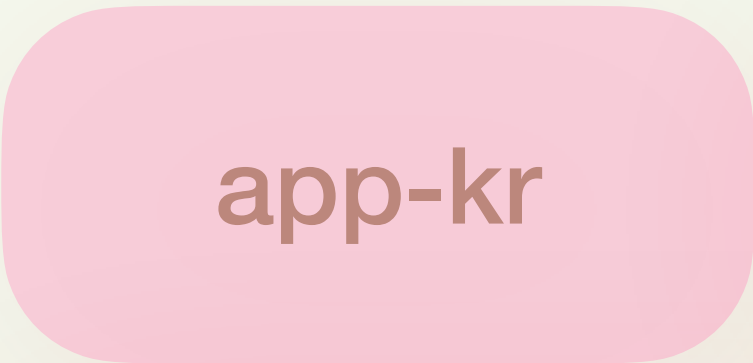
React Utility Layer

Lower Layer

Packages

모노레포 레이어 아키텍처

Upper Layer



Application Layer



Domain Layer



Service Layer



UI Layer



React Utility Layer

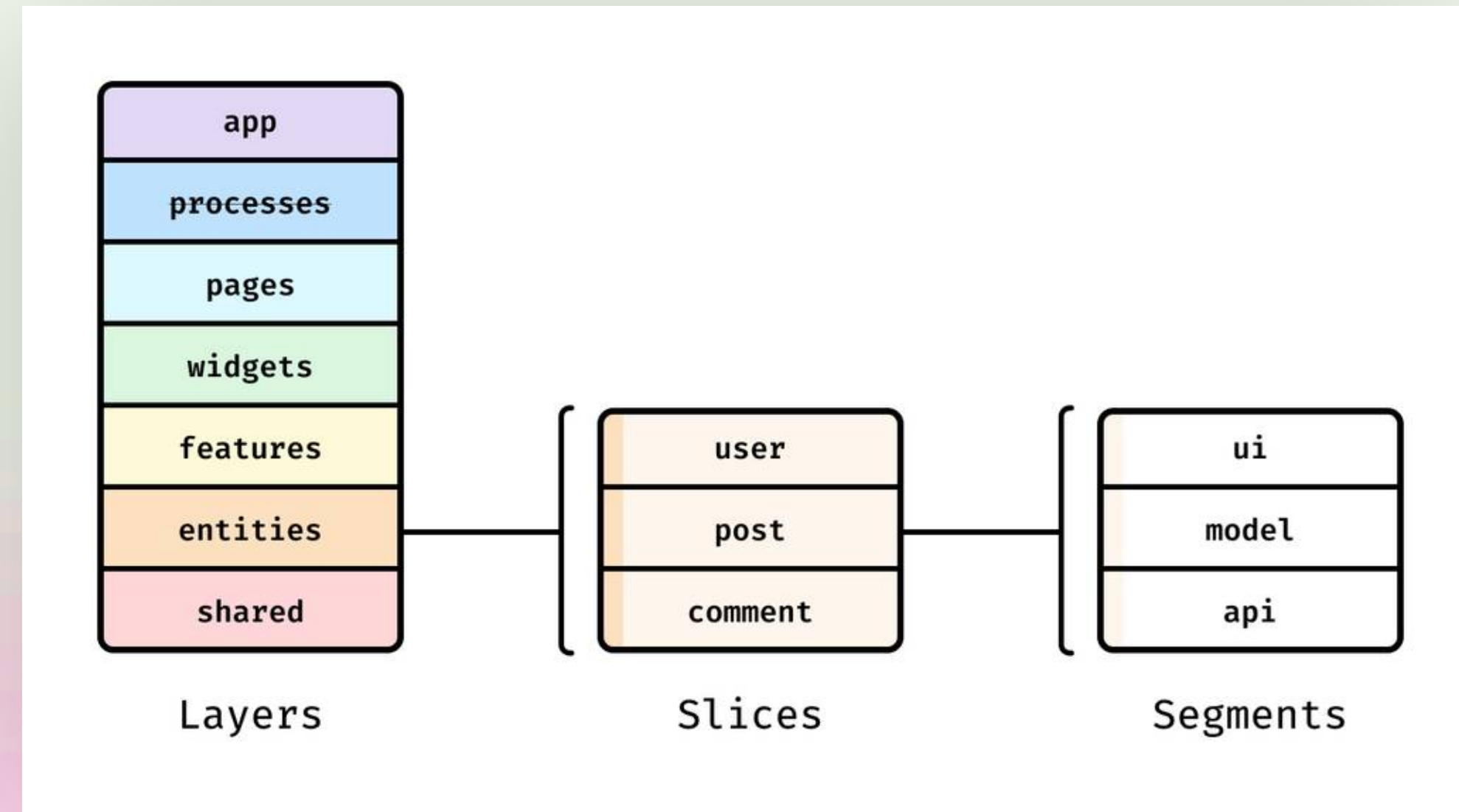
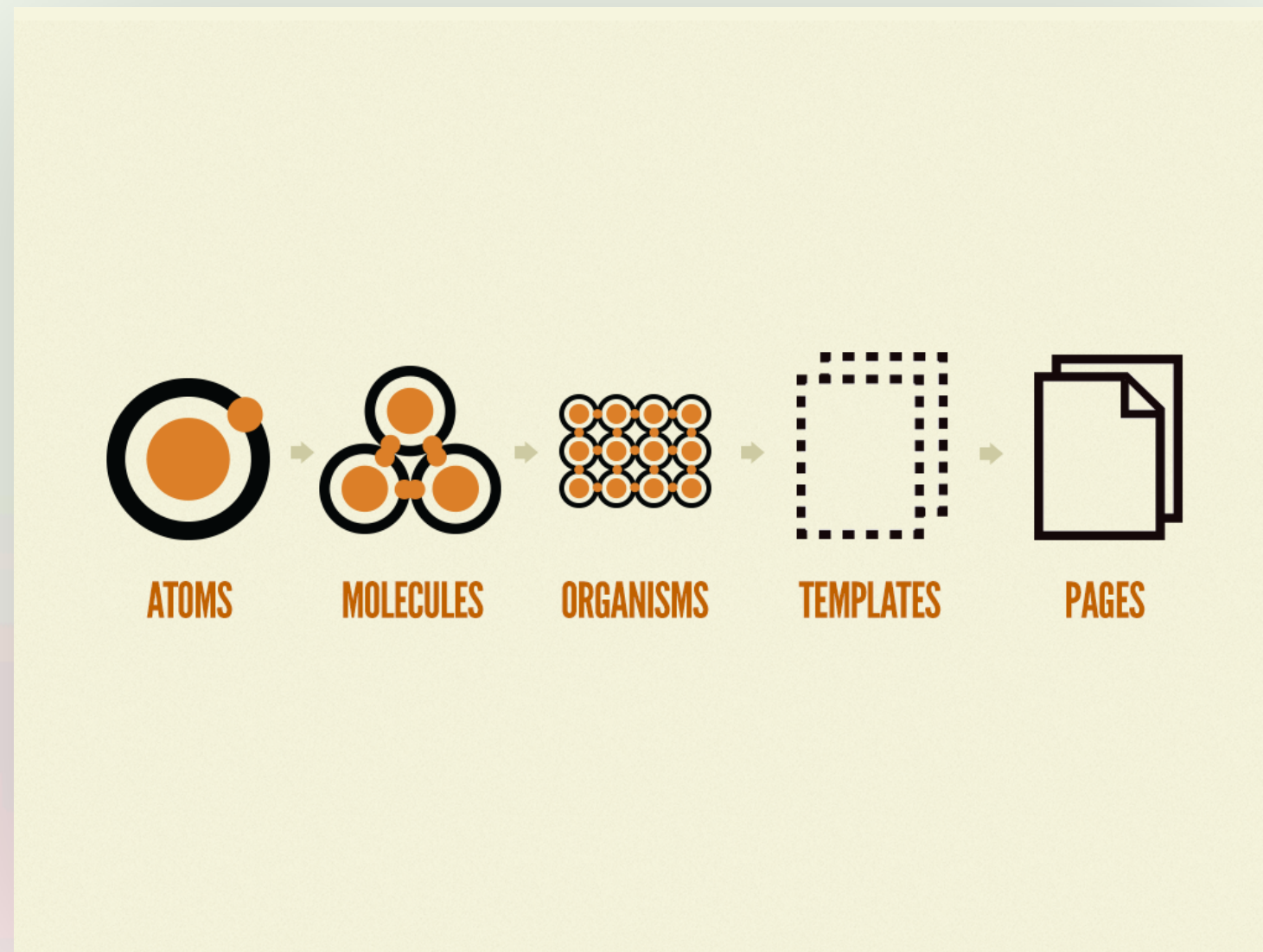
Lower Layer



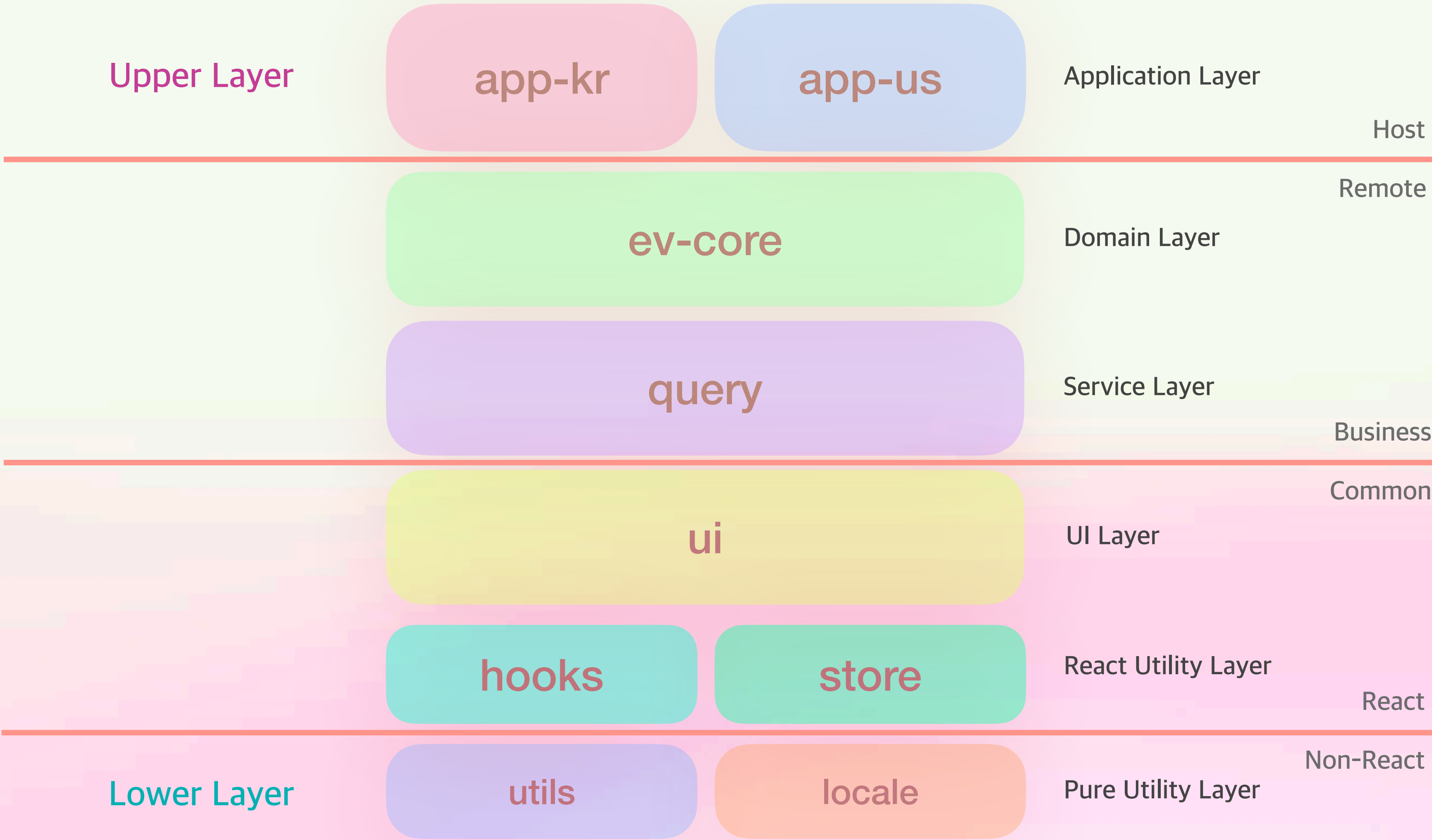
Pure Utility Layer

과연 남들이 봤을때 직관적이라고 할 수 있을까?

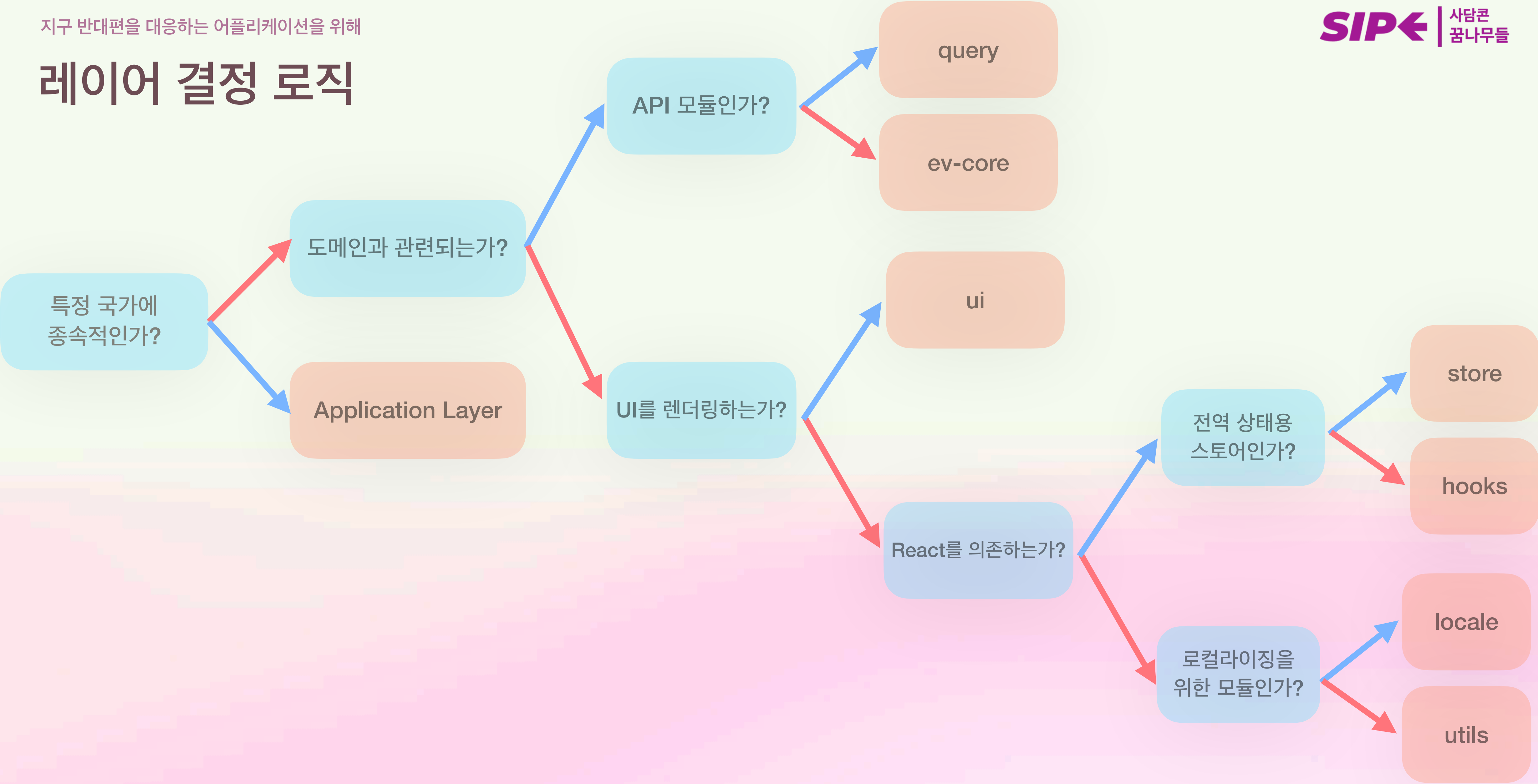
우리는 이미 레이어 아키텍처에 대한 피로감이 있습니다



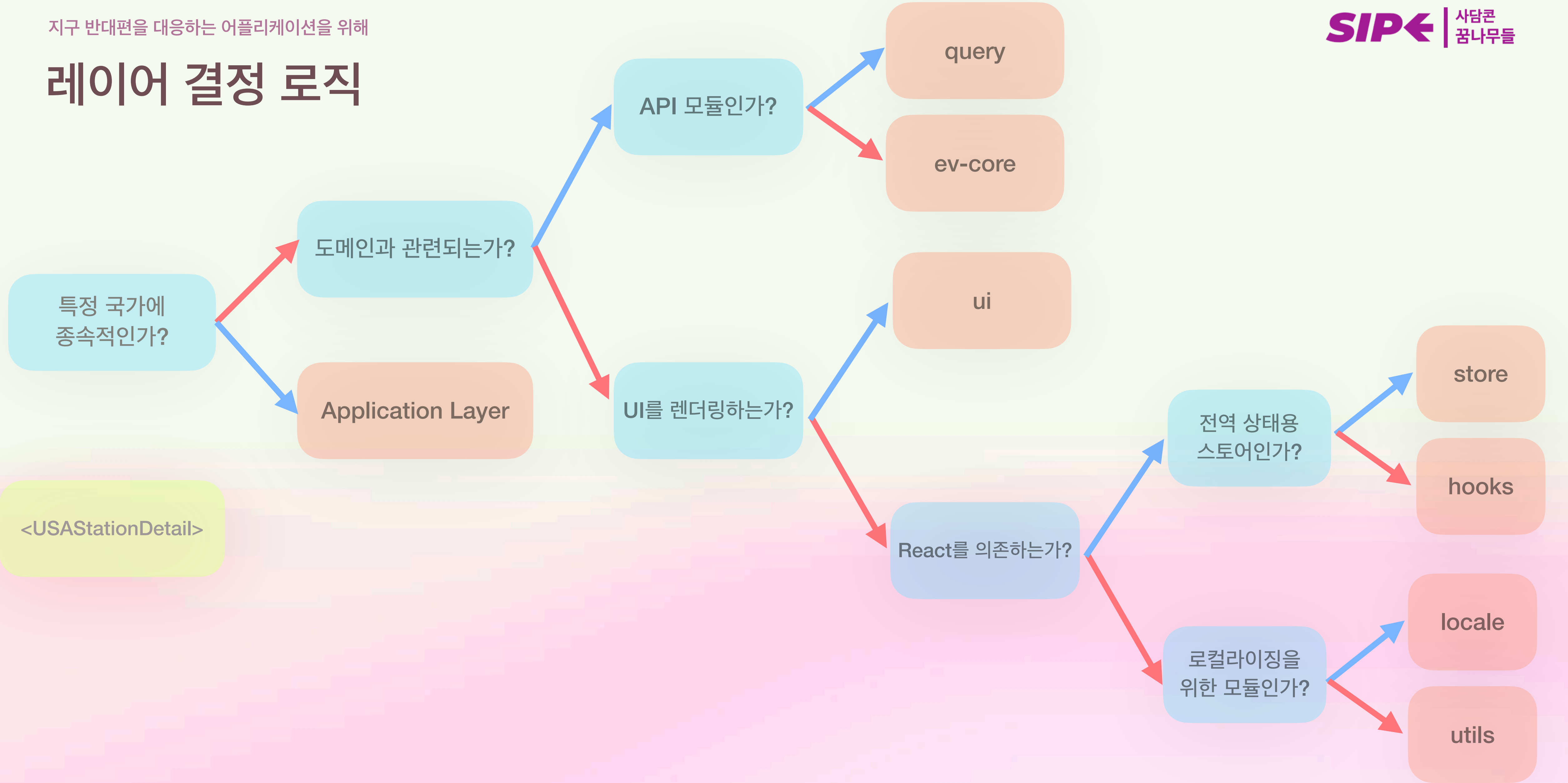
모노레포 레이어 아키텍처



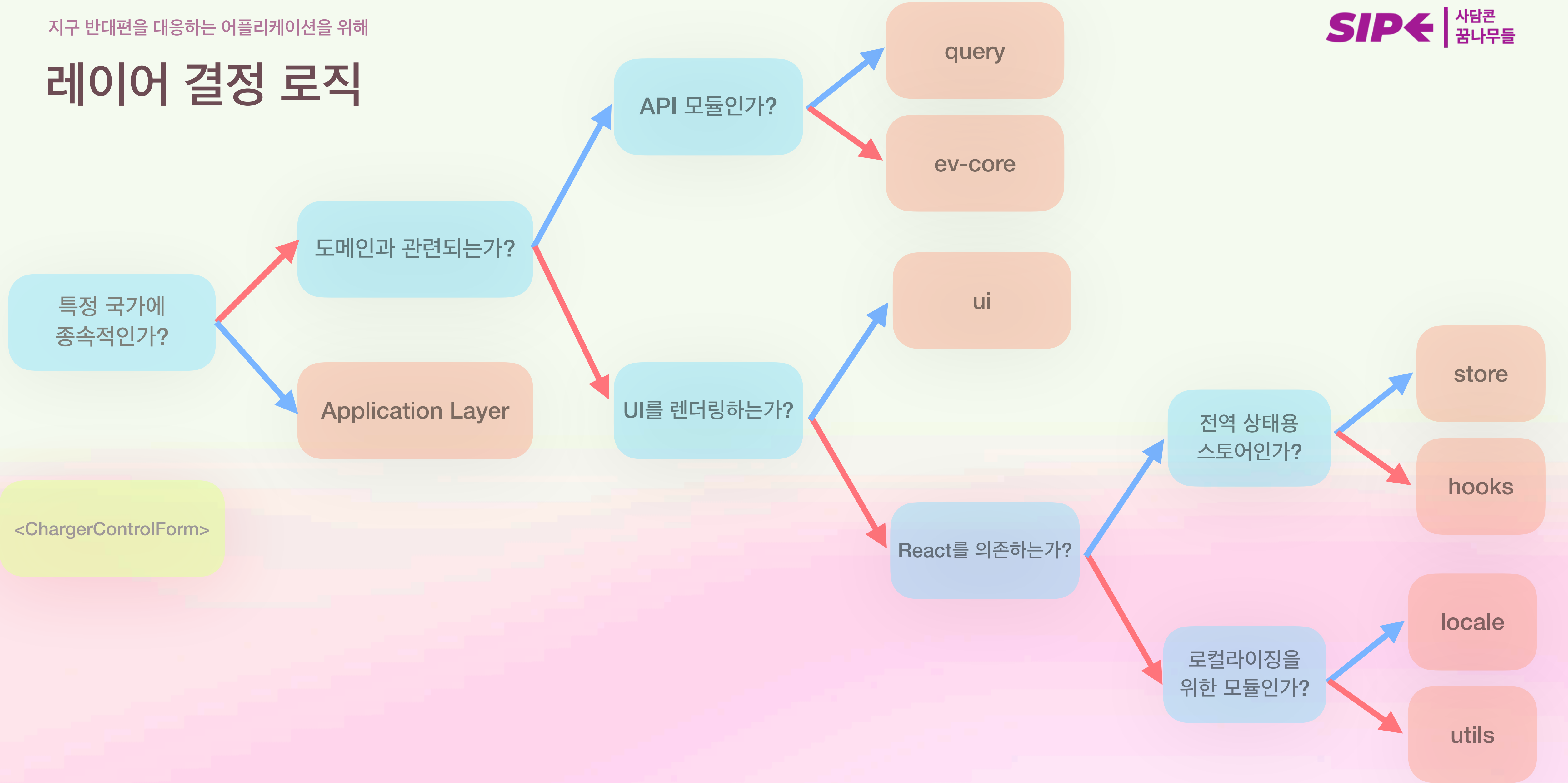
레이어 결정 로직



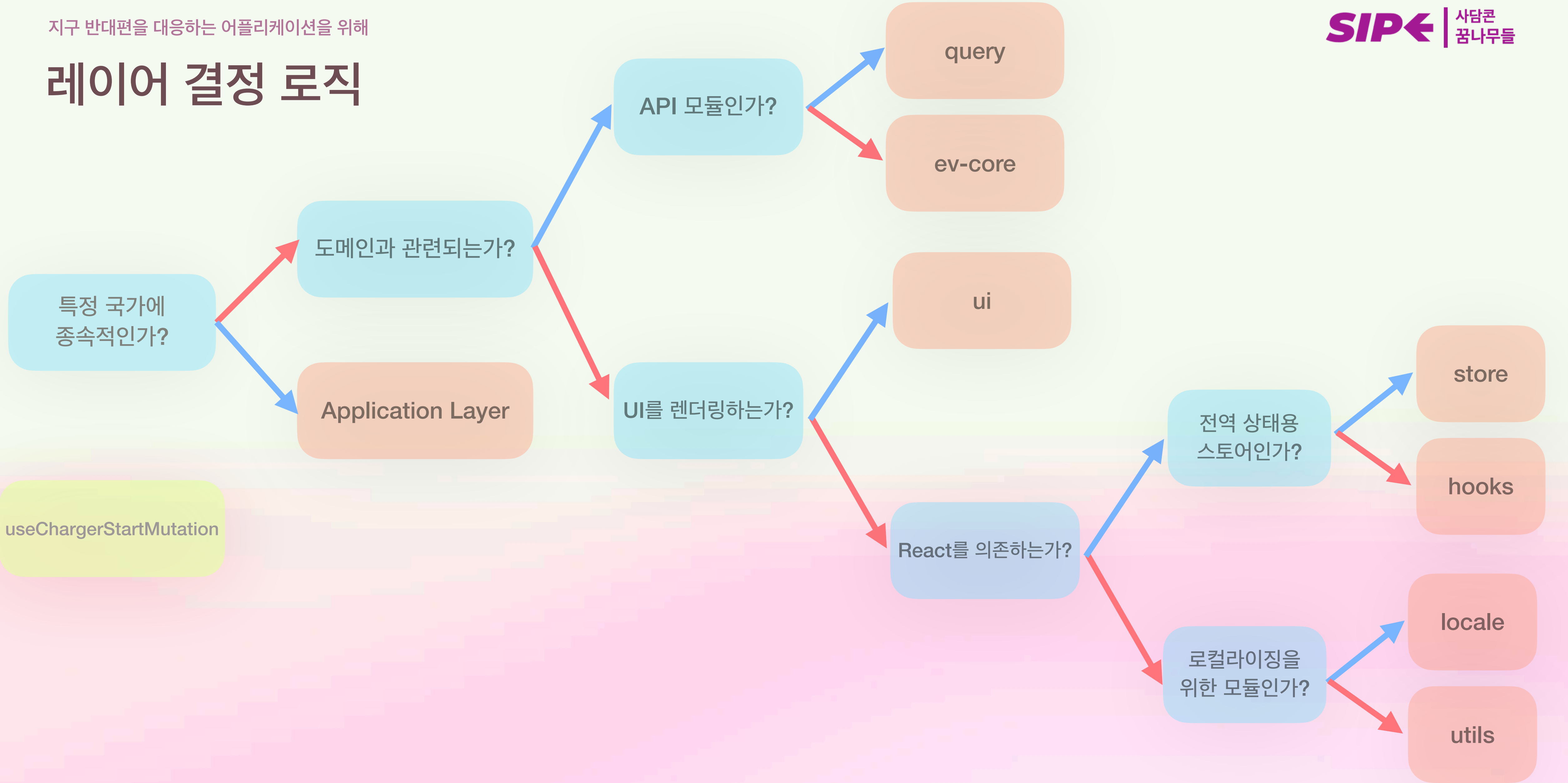
레이어 결정 로직



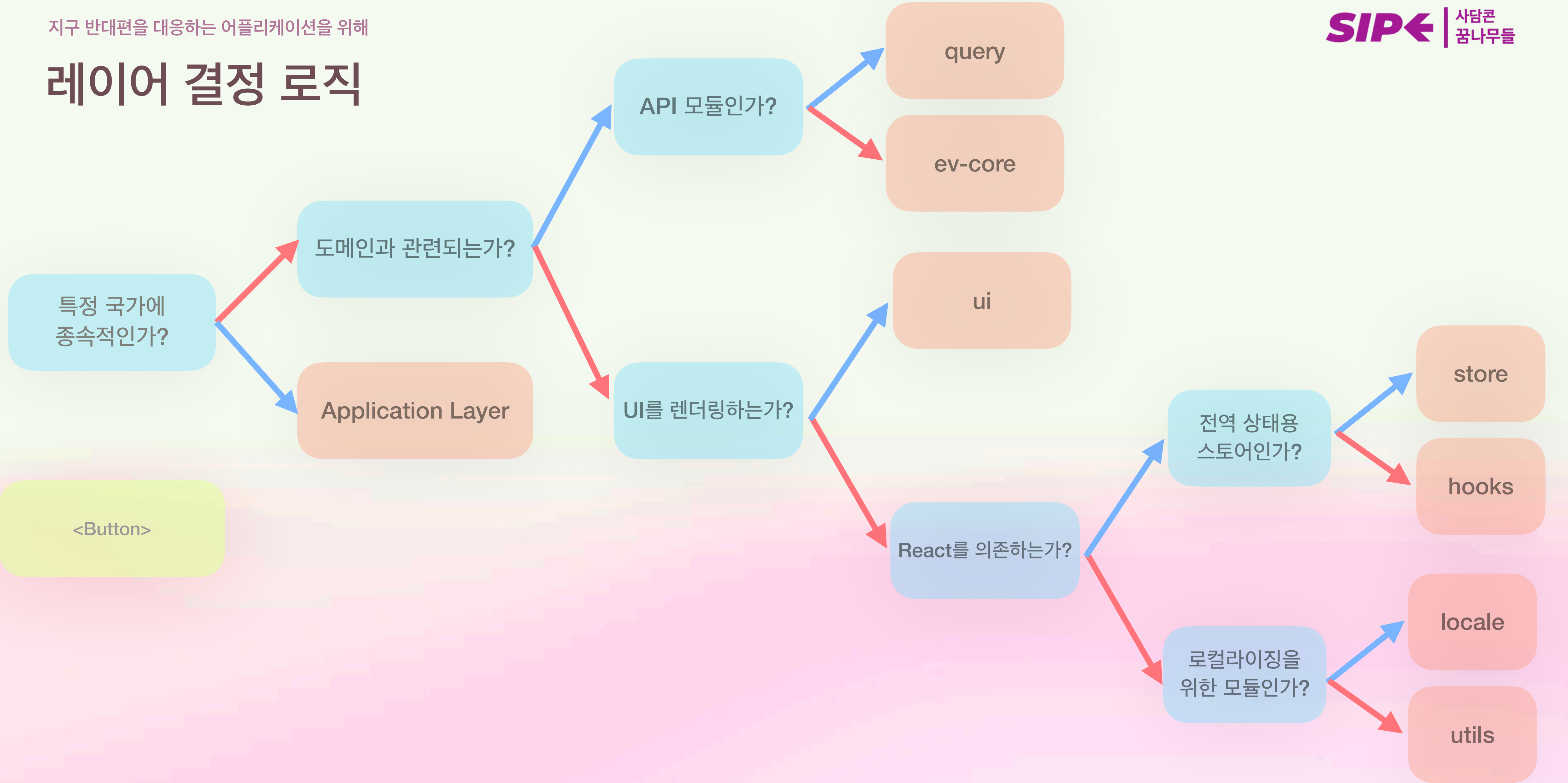
레이어 결정 로직



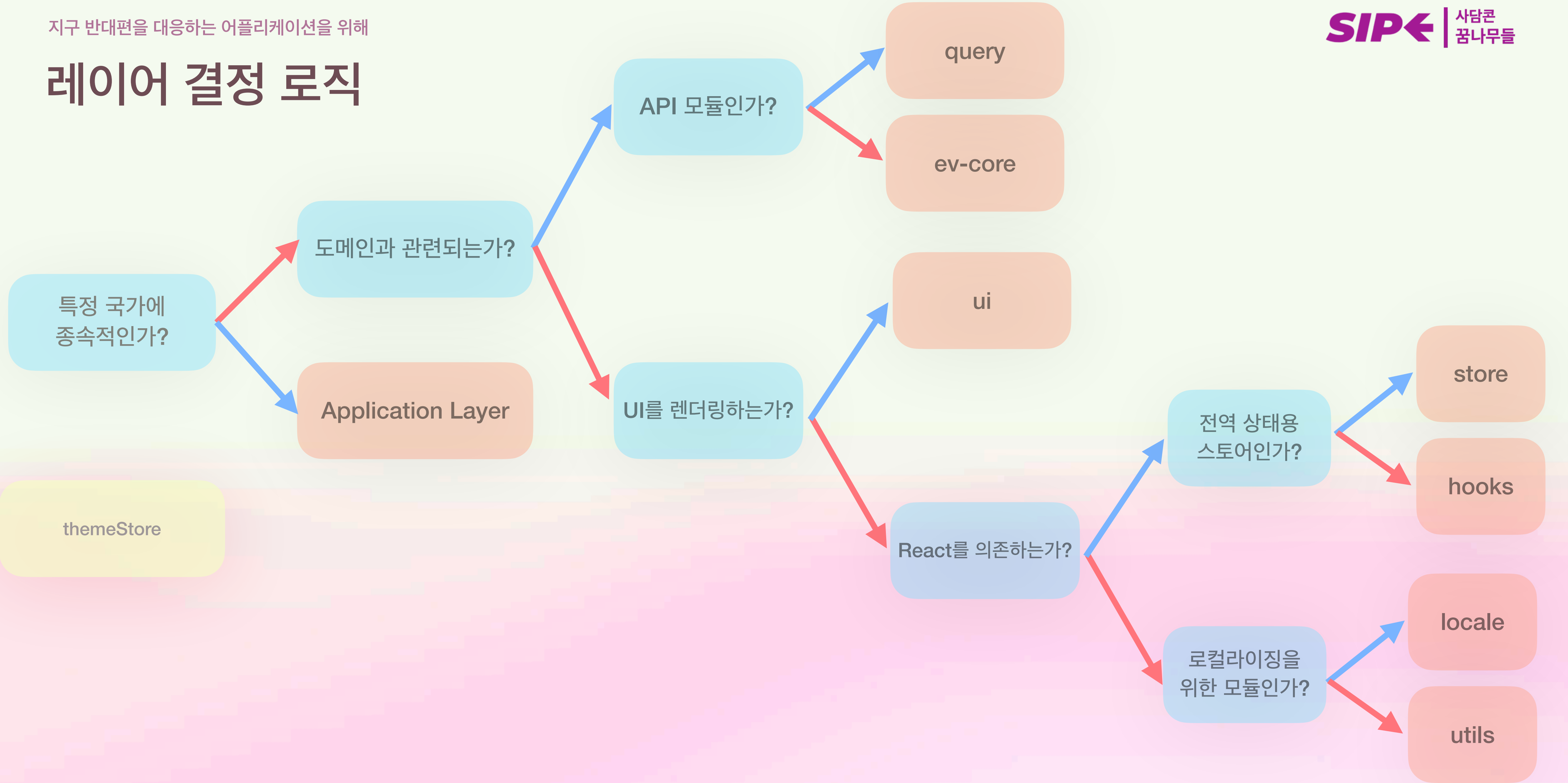
레이어 결정 로직



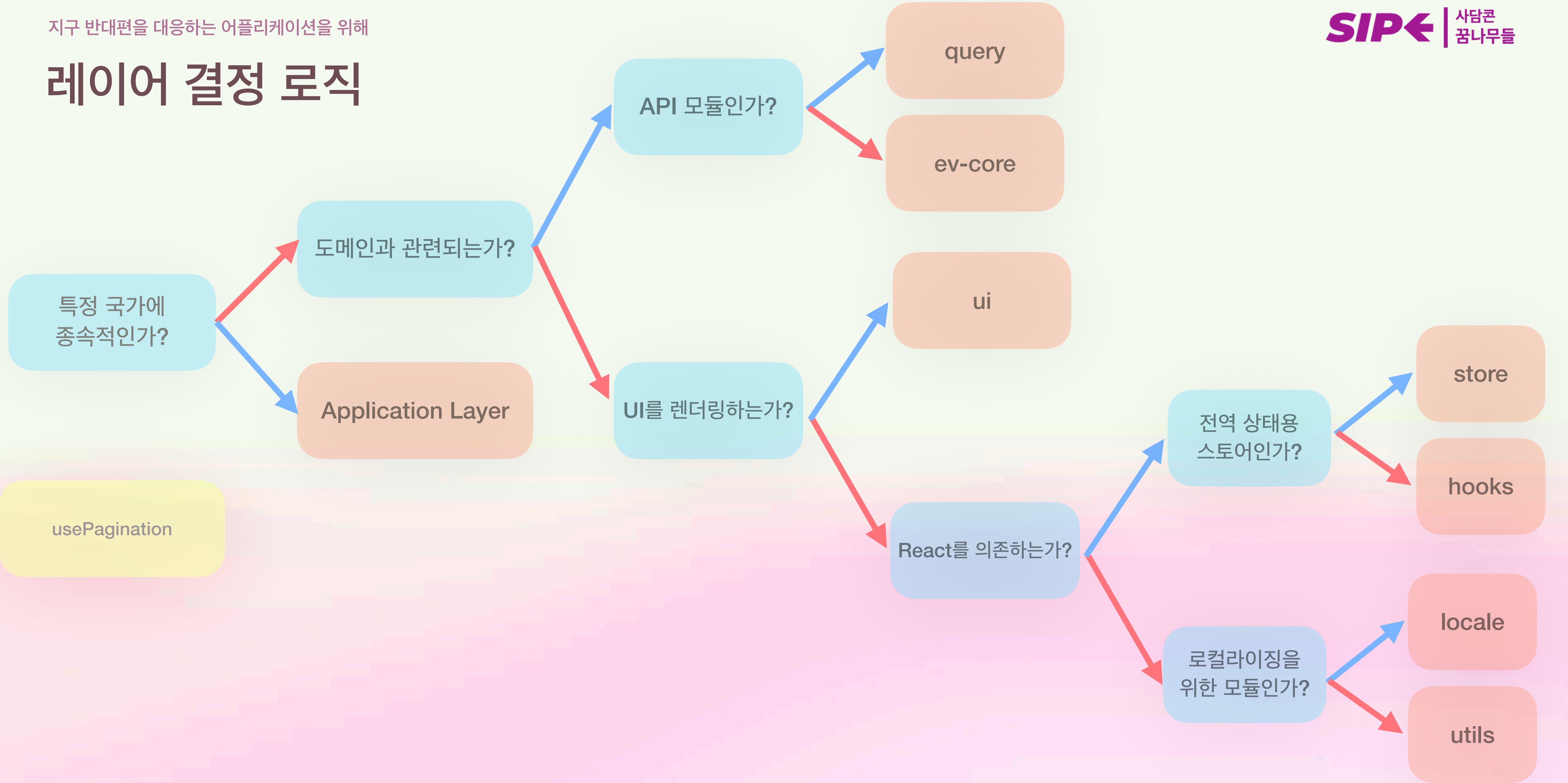
레이어 결정 로직



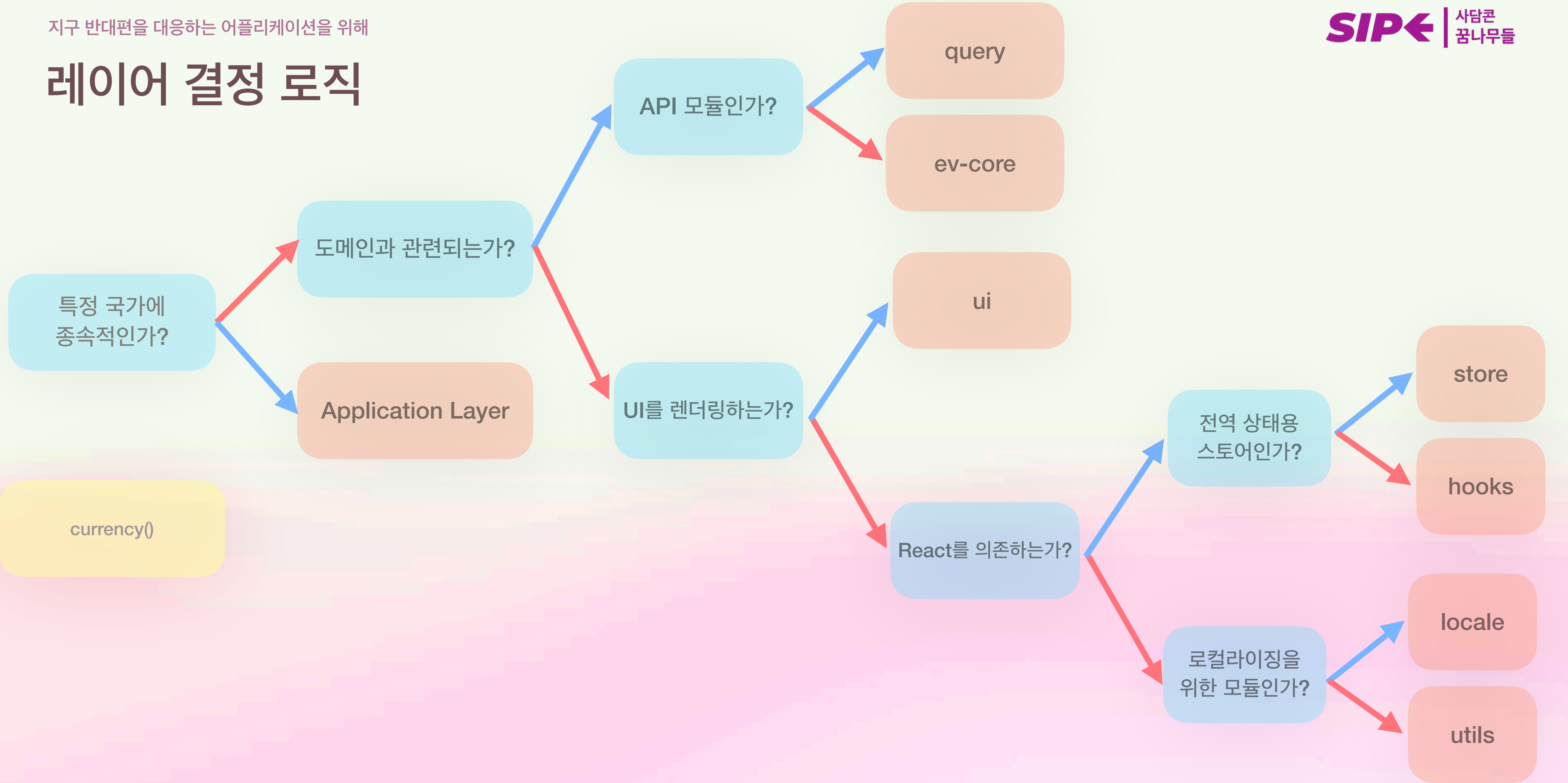
레이어 결정 로직



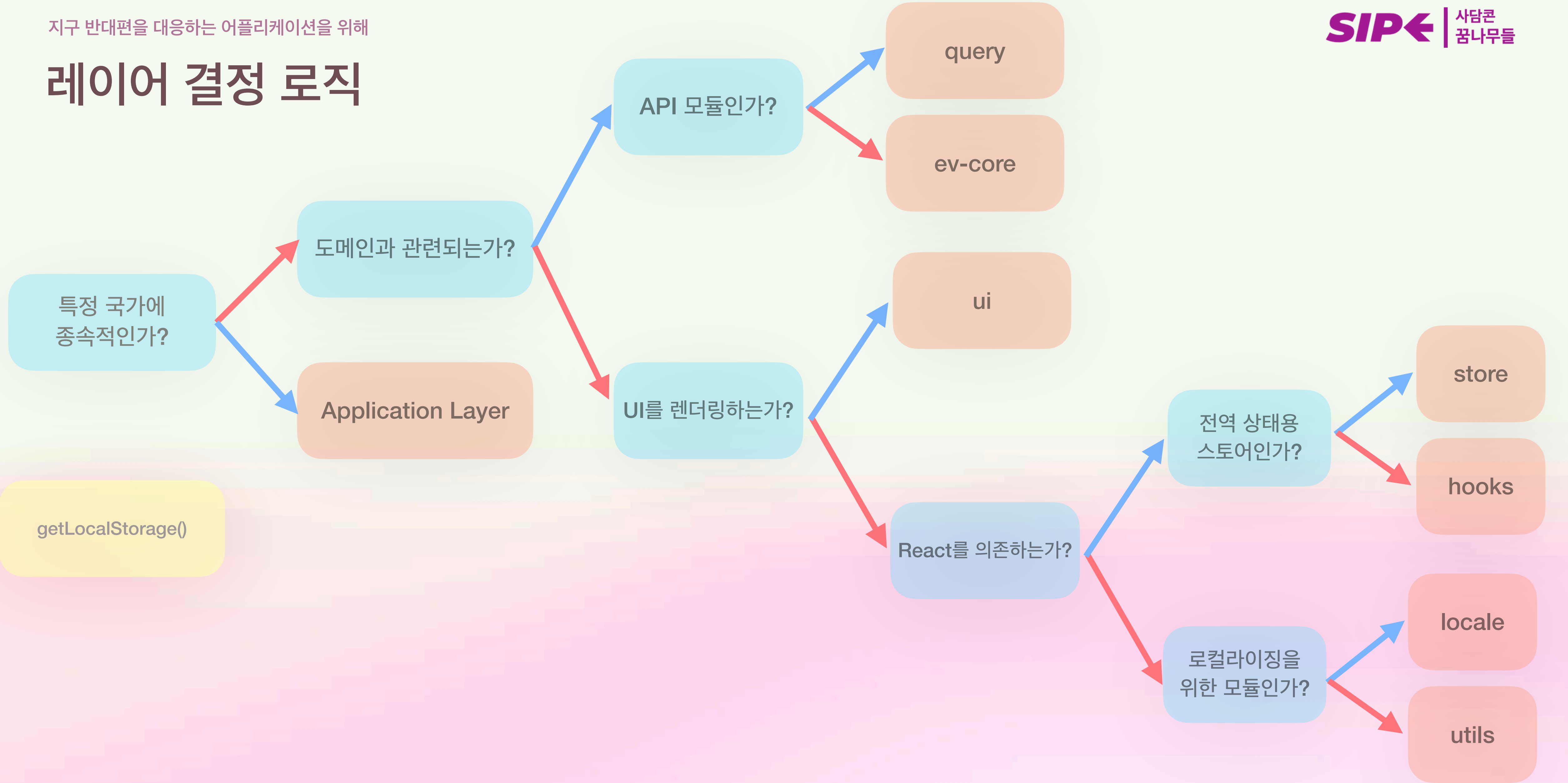
레이어 결정 로직



레이어 결정 로직

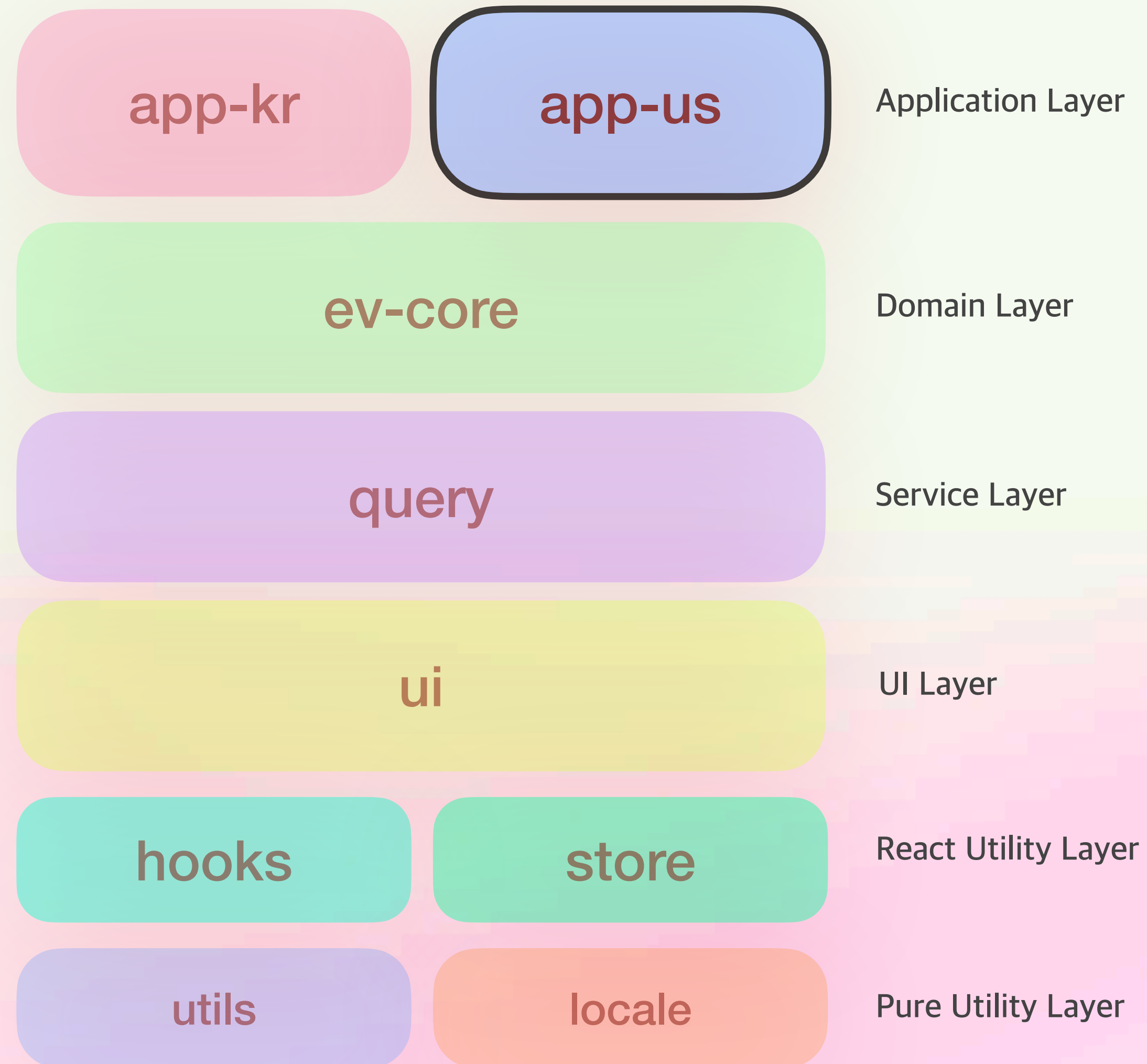


레이어 결정 로직



지구 반대편을 대응하는 어플리케이션을 위해

의존 레이어

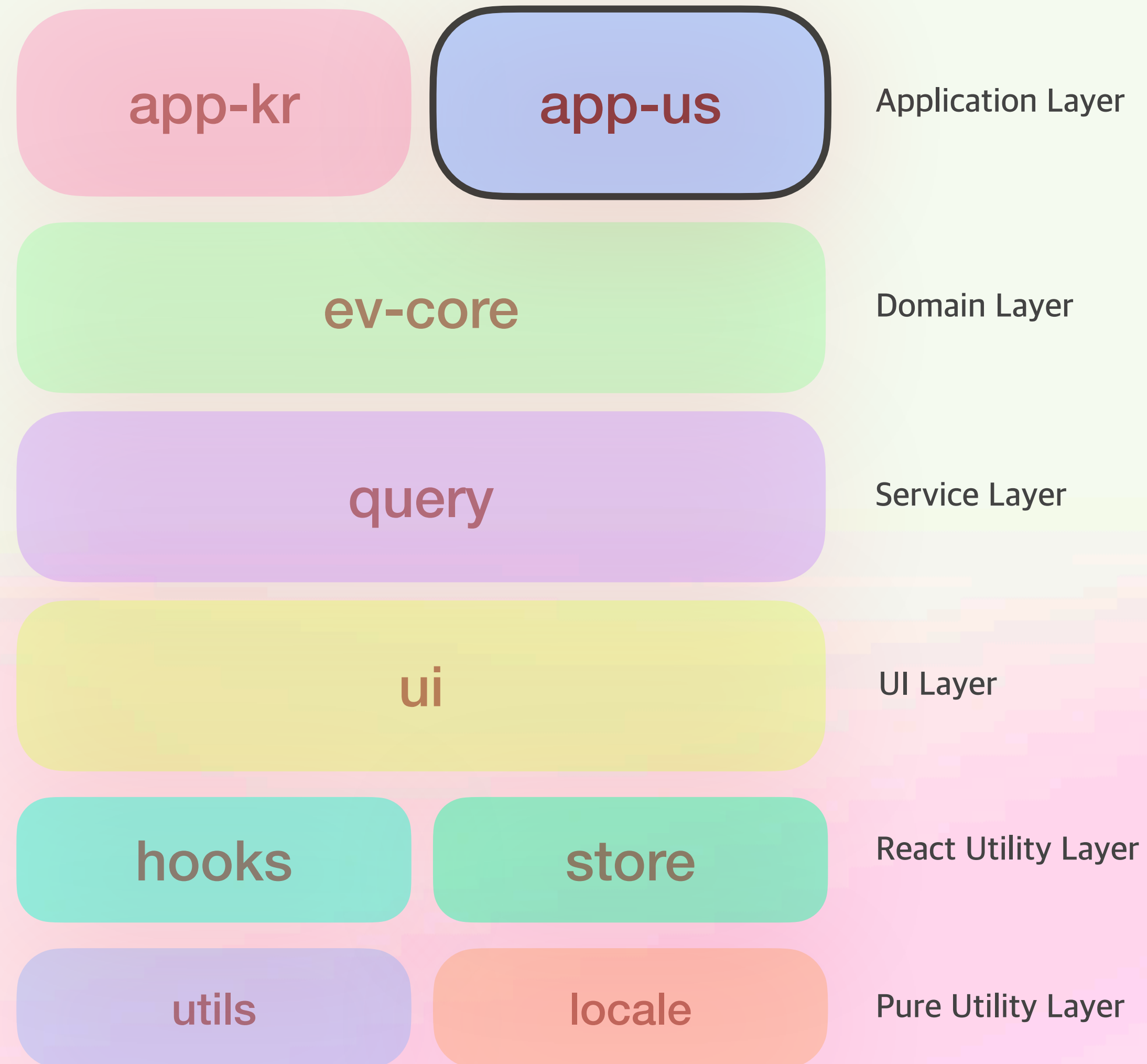


```
// apps/csms-us/pages/USAStationPage.tsx
```

```
function USAStationPage() {  
  const { station_id } = useQueryString();  
  
  return (  
    <div>  
      <USAStationDetail stationID={station_id} />  
    </div>  
  )  
}
```


지구 반대편을 대응하는 어플리케이션을 위해

의존 레이어

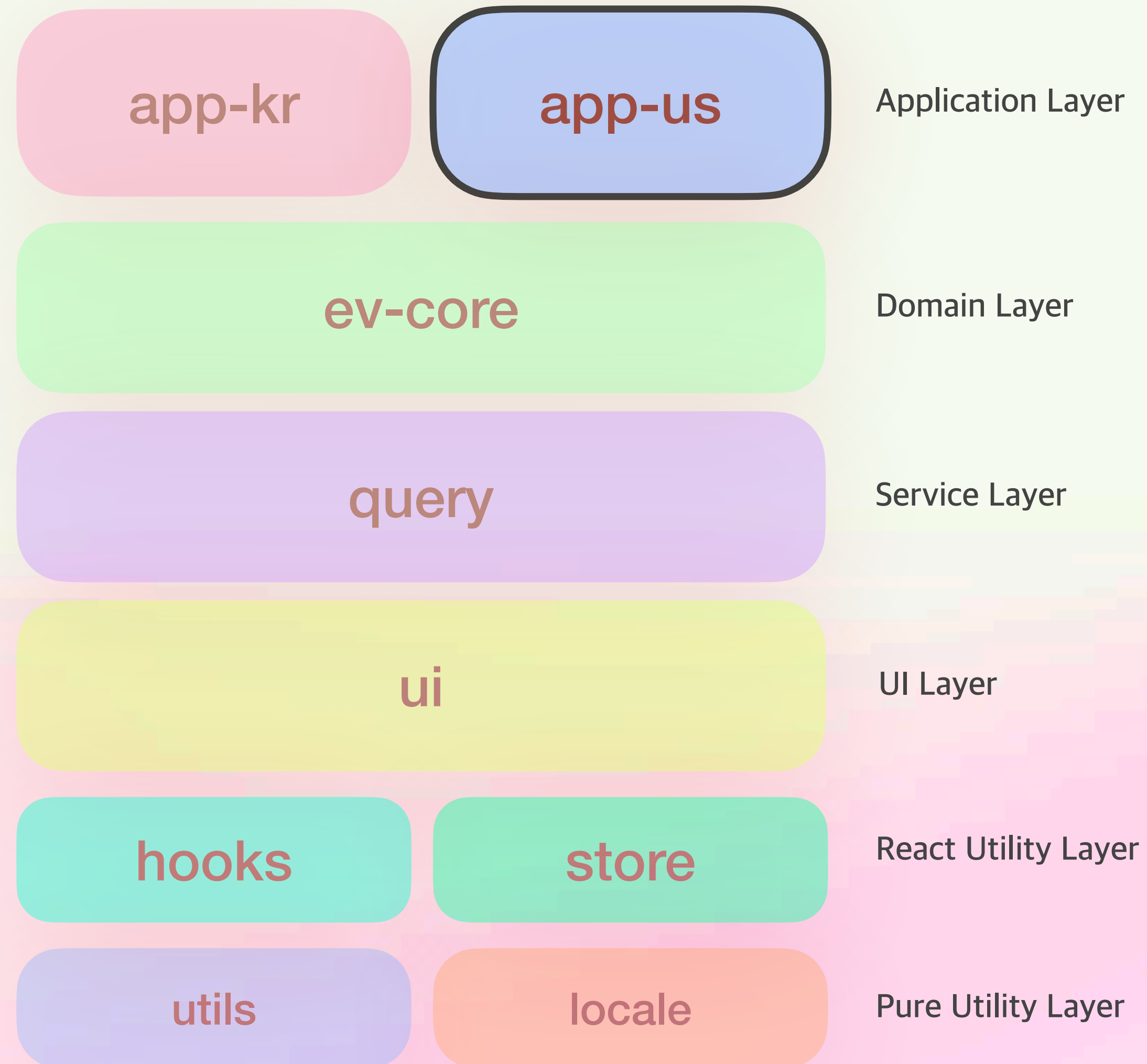


```
// apps/csms-us/pages/USAStationPage.tsx
```

```
function USAStationPage() {  
  const { station_id } = useQueryString();  
  
  return (  
    <div>  
      <USAStationDetail stationID={station_id} />  
    </div>  
  )  
}
```

지구 반대편을 대응하는 어플리케이션을 위해

의존 레이어

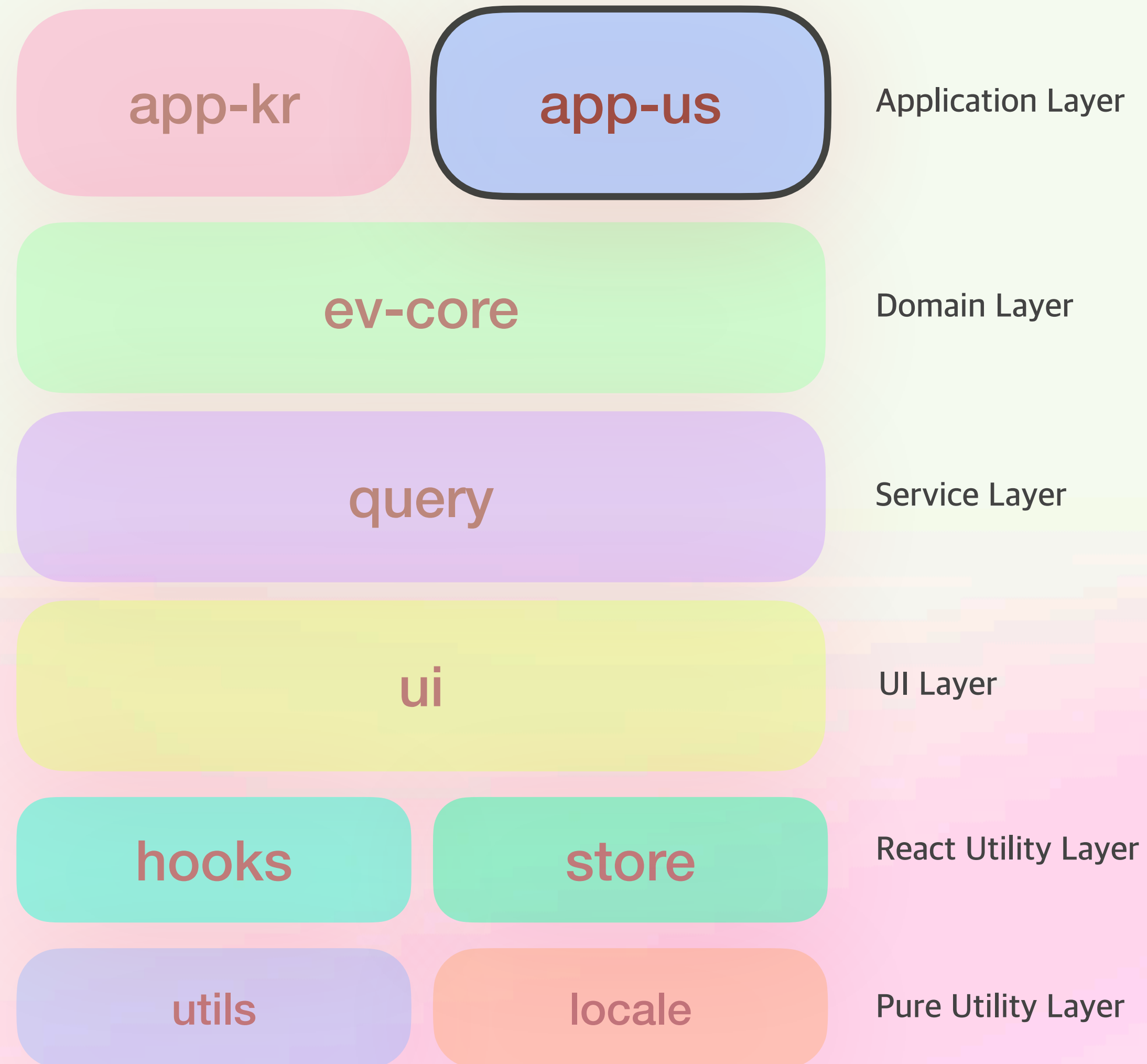


```
// apps/csms-us/components/USASStationDetail.tsx
```

```
function USASStationDetail({ station_id }: { station_id: string }) {  
  const { data: station } = useUSASStationDetailQuery();  
  
  return (  
    <div>  
      <h1>{station.name}</h1>  
      <h1>{station.region}</h1>  
      // ...  
    </div>  
  )  
}
```

지구 반대편을 대응하는 어플리케이션을 위해

의존 레이어

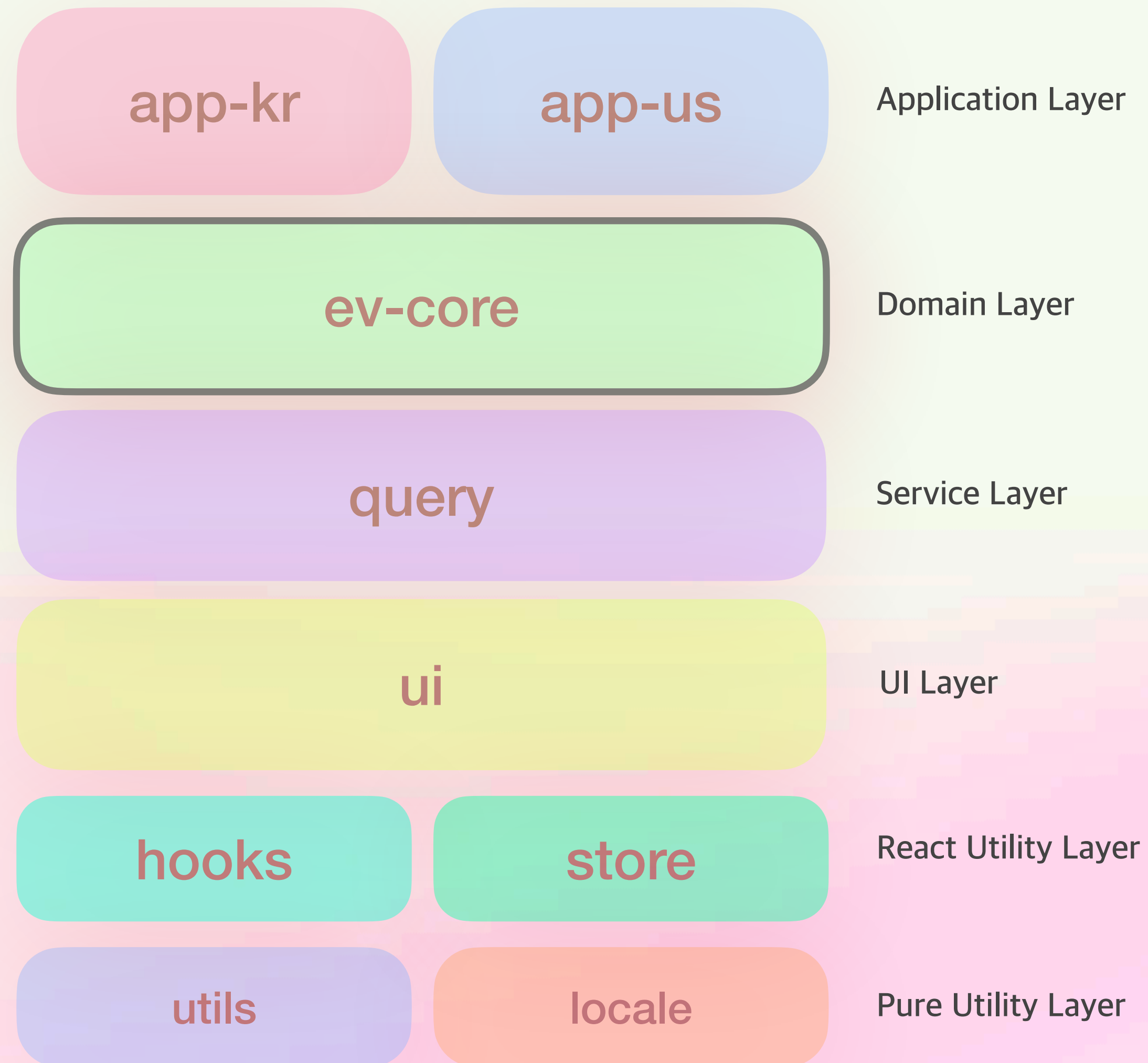


```
// apps/csms-us/pages/ChargerContronPage.tsx
```

```
function ChargerContronPage() {  
  const { charger_id } = useQueryString();  
  
  return (  
    <div>  
      <ChargerControlForm chargerID={charger_id} />  
    </div>  
  )  
}
```


지구 반대편을 대응하는 어플리케이션을 위해

의존 레이어



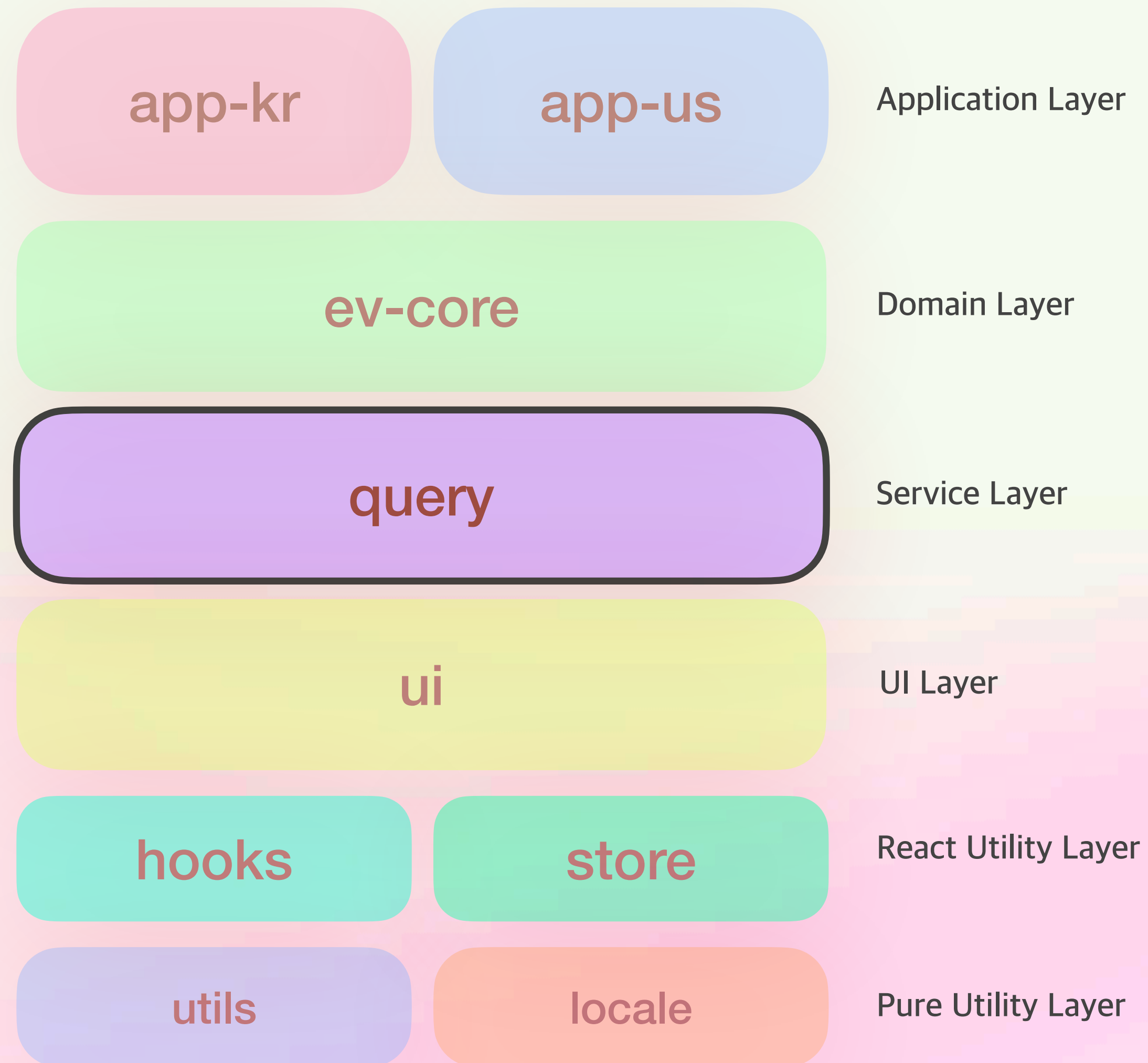
```
// apps/csms-us/pages/ChargerContronPage.tsx
```

```
function ChargerContronPage() {
  const { charger_id } = useQueryString();

  return (
    <div>
      <ChargerControlForm chargerID={charger_id} />
    </div>
  )
}
```

지구 반대편을 대응하는 어플리케이션을 위해

의존 레이어

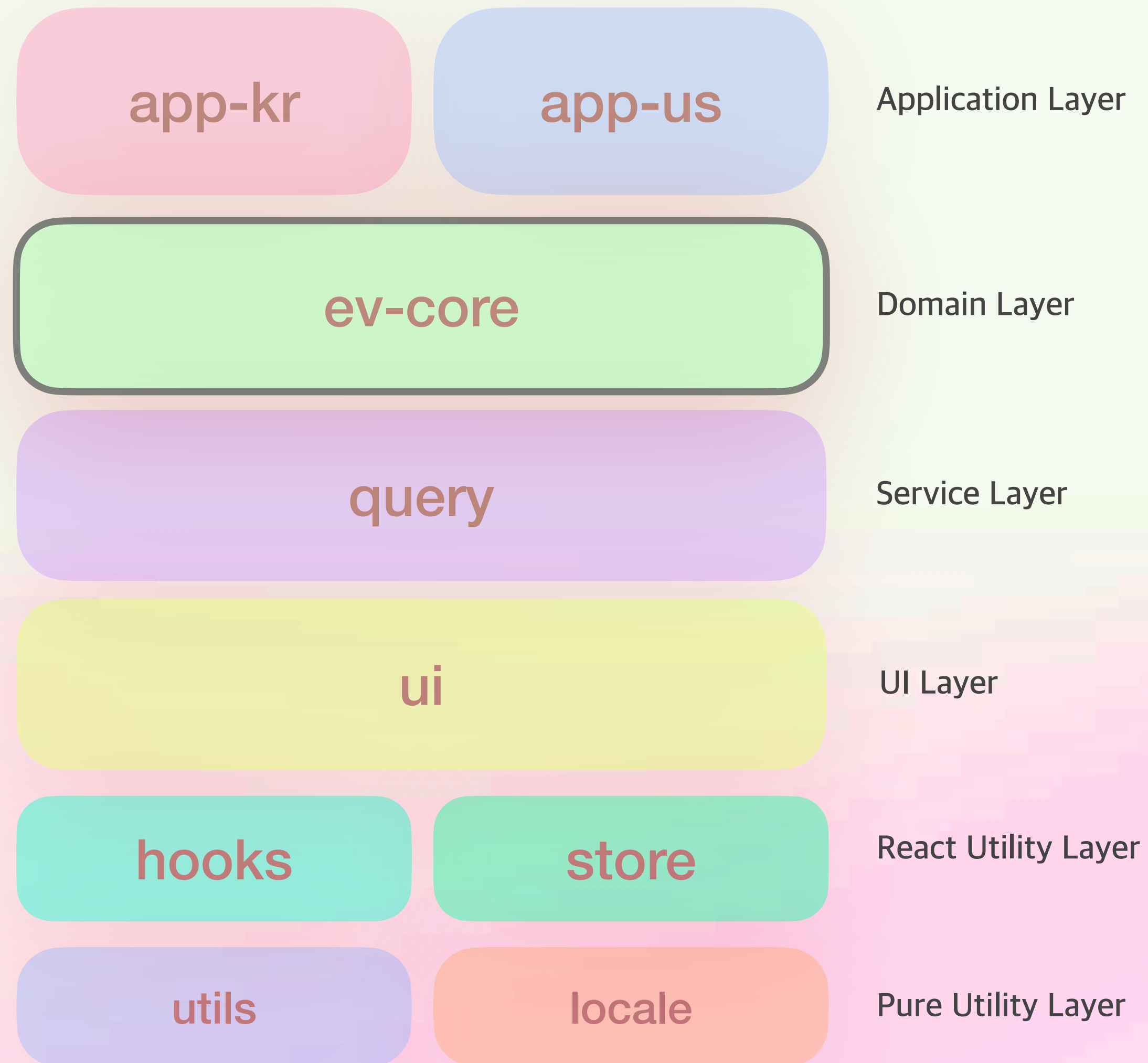


```
// packages/ev-core/components/ChargerControlForm.tsx
```

```
function ChargerControlForm({ charger_id }: { charger_id: string }) {  
  const { mutate } = useChargerControlMutate({ charger_id });  
  
  return (  
    <div>  
      // ...  
      <Button onClick={mutate}>Cotnrol</button>  
    </div>  
  )  
}
```


지구 반대편을 대응하는 어플리케이션을 위해

의존 레이어

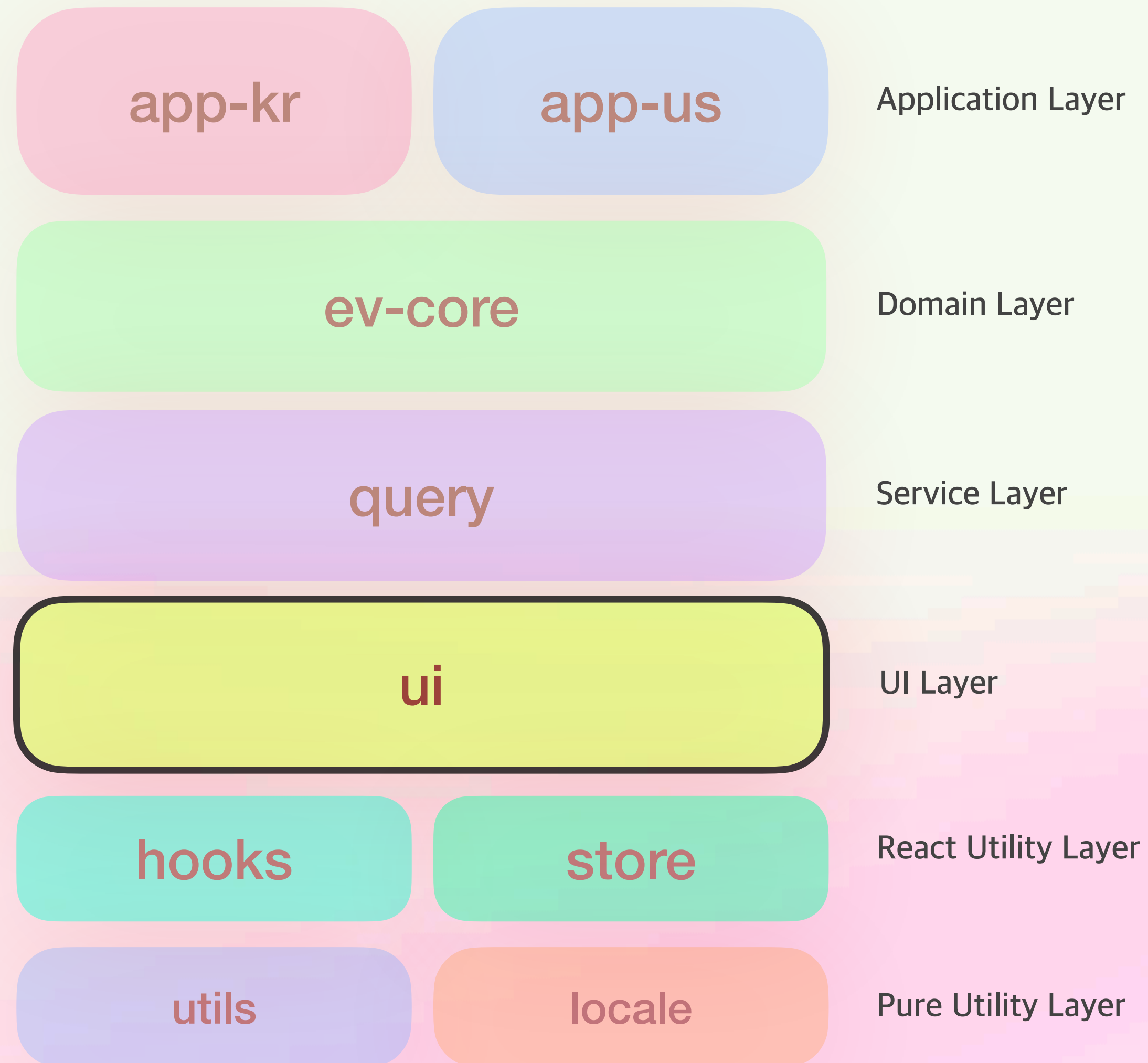


```
// packages/ev-core/components/ChargerControlForm.tsx
```

```
function ChargerControlForm({ charger_id }: { charger_id: string }) {  
  const { mutate } = useChargerControlMutate({ charger_id });  
  
  return (  
    <div>  
      // ...  
      <Button onClick={mutate}>Cotnrol</button>  
    </div>  
  )  
}
```


지구 반대편을 대응하는 어플리케이션을 위해

의존 레이어

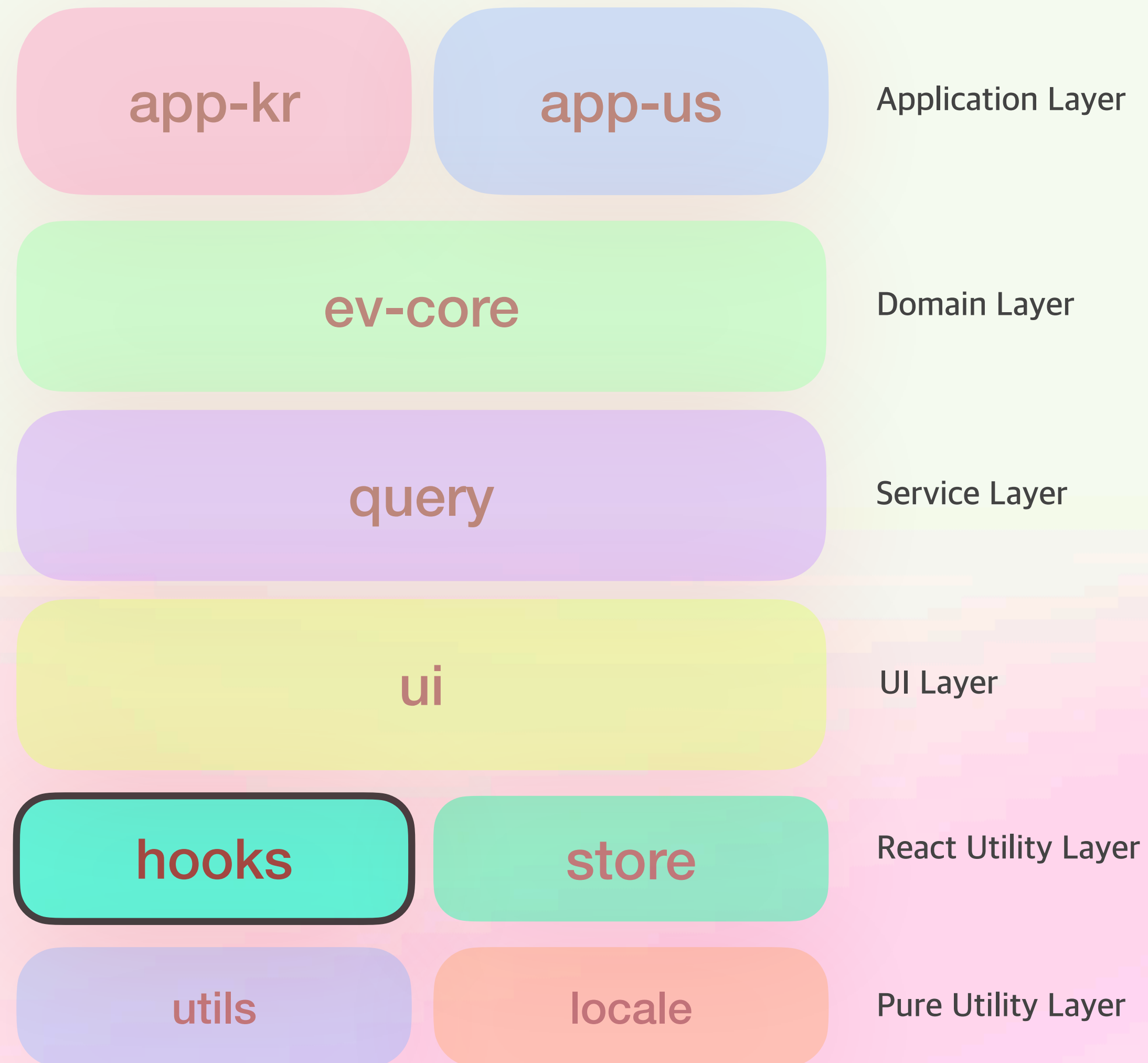


```
// packages/ev-core/components/ChargerControlForm.tsx
```

```
function ChargerControlForm({ charger_id }: { charger_id: string }) {  
  const { mutate } = useChargerControlMutate({ charger_id });  
  
  return (  
    <div>  
      // ...  
      <Button onClick={mutate}>Cotnrol</button>  
    </div>  
  )  
}
```

지구 반대편을 대응하는 어플리케이션을 위해

의존 레이어

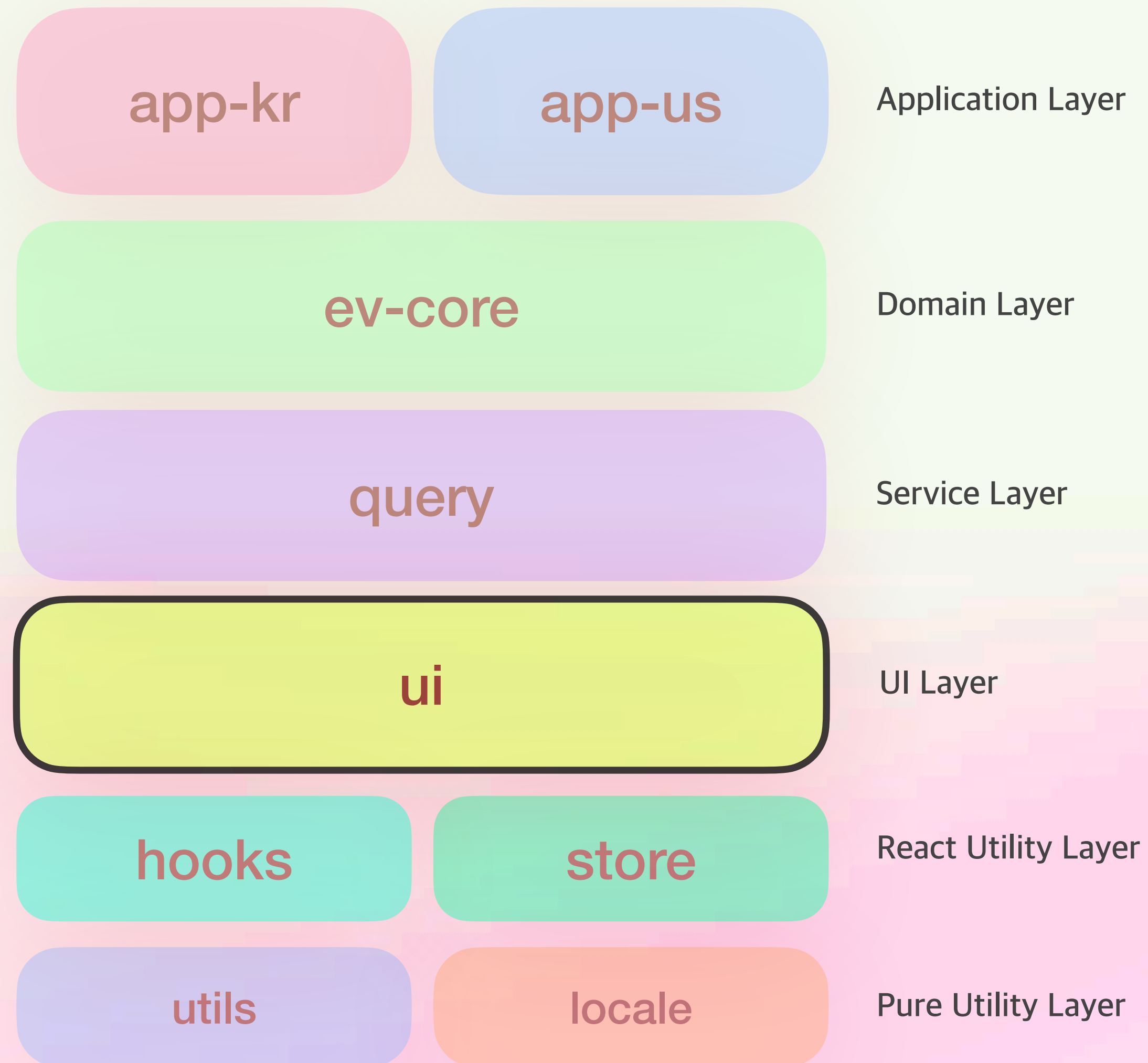


```
// apps/ui/Button.tsx
```

```
function Button(props: ButtonProps) {  
  const { value, defaultValue } = useUncontrolled(props);  
  
  return (  
    <button {...props}>  
      {props.children}  
    </div>  
  )  
}
```

지구 반대편을 대응하는 어플리케이션을 위해

의존 레이어

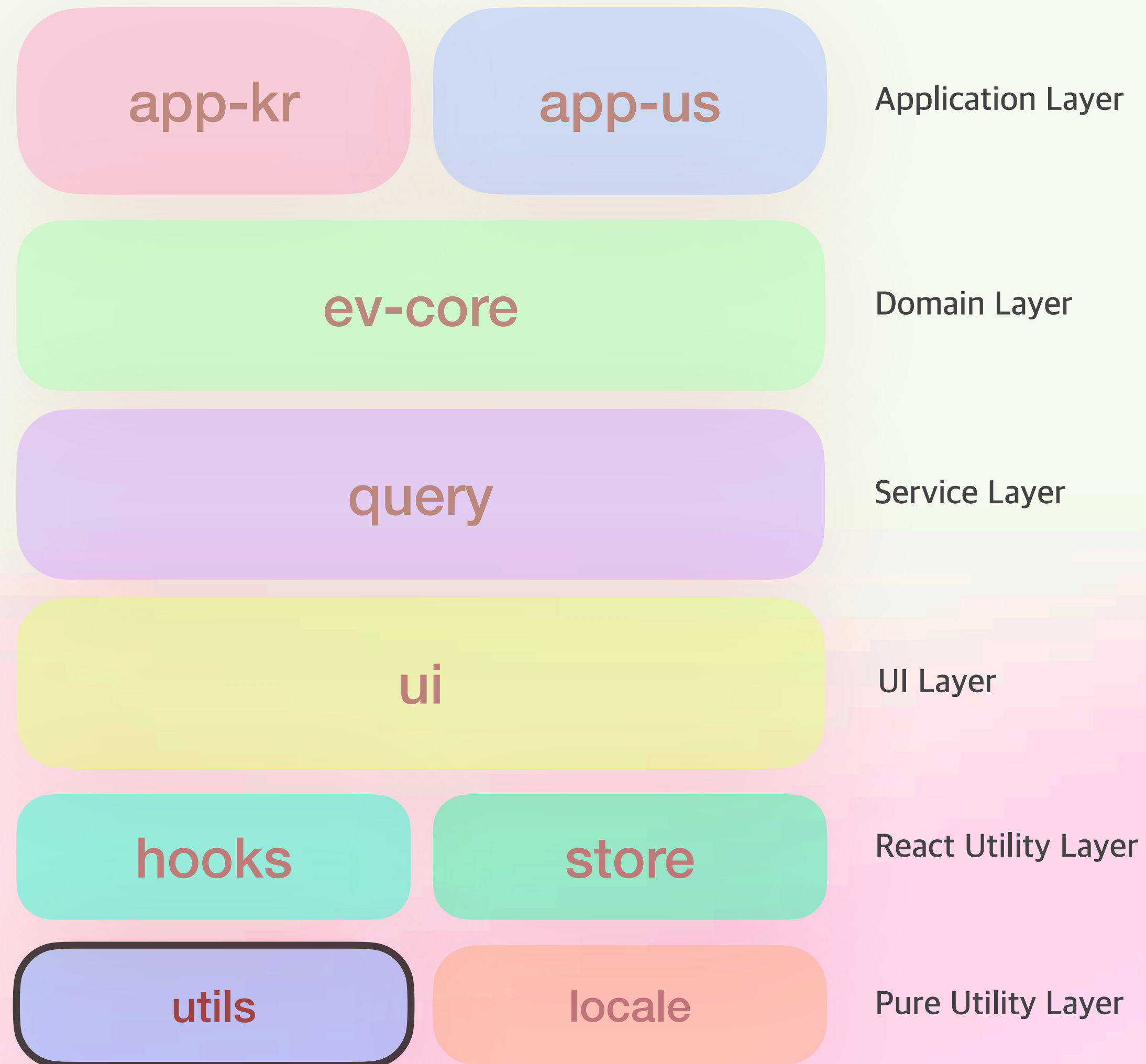


```
// packages/ui/Button.tsx
```

```
function Button(props: ButtonProps) {  
  const { value, defaultValue } = useUncontrolled(props);  
  
  return (  
    <button {...props} >  
      {props.children}  
    </div>  
  )  
}
```


지구 반대편을 대응하는 어플리케이션을 위해

의존 레이어



```
// packages/ui/Button.tsx
```

```
function Button(props: ButtonProps) {  
  const { value, defaultValue } = useUncontrolled(props);  
  
  return (  
    <button {...props}  
      onClick={{(e) => {  
        loggingEvent(e);  
        props?.onChange(e);  
      }}}  
    >  
      {props.children}  
    </div>  
  )  
}
```


요구사항 변동에 따른 레이어 이동

변경전



그 컴포넌트가 어디있더라...



미국에서만 쓰이던 xx 기능
한국에도 적용하기로 했어요

변경후



app-us



ev-core

내겐 너무 과분한 도메인 용어

Plan은 플랜인가요 요금제인가요?

충전소 ID, 스테이션 ID, CS ID

모델은 모델과 제품, 어떤걸 써야하나요?

내겐 너무 과분한 도메인 용어



반복되는 휴먼에러와 함께 추락하는 UX 일관성

유비쿼터스 언어

유비쿼터스 언어



유비쿼터스 언어 Ubiquitous Language

도메인 주도 개발에서의 핵심 개념 중 하나로
개발자와 도메인 전문가와의 의사소통에서
효율적이고 정확한 소통을 위한 공통 언어 시스템

지구 반대편을 대응하는 어플리케이션을 위해

유비쿼터스 언어



고객은 책을 장바구니에 담고,
주문을 완료하면 결제가 되고 배송이 시작돼요.

유비쿼터스 언어



고객은 책을 장바구니에 담고,
주문을 완료하면 결제가 되고 배송이 시작돼요.

Customer	주문을 생성하는 사람
Book	주문할 수 있는 상품
Cart	고객이 주문 전에 책을 모아두는 공간
Order	고객이 결제한 책의 묶음
Payment	주문을 확정하기 위한 결제 행위
Shipment	주문이 완료된 후 책이 이동하는 과정

Ubiquitous Language Pack

지구 반대편을 대응하는 어플리케이션을 위해

실제 비즈니스 관계에 의존하는
유비쿼터스 언어 팩을 사용하여
재사용 가능한 용어집을 코드로 구현하기

```
interface Station {  
    station_id: number;  
    station_name: string;  
    city: string;  
    region: string;  
    // ...  
}
```

지구 반대편을 대응하는 어플리케이션을 위해

실제 비즈니스 관계에 의존하는
유비쿼터스 언어 팩을 사용하여
재사용 가능한 용어집을 코드로 구현하기

```
interface Station {  
    station_id: number;  
    station_name: string;  
    city: string;  
    region: string;  
    // ...  
}
```

지구 반대편을 대응하는 어플리케이션을 위해

실제 비즈니스 관계에 의존하는
유비쿼터스 언어 팩을 사용하여
재사용 가능한 용어집을 코드로 구현하기

```
const STATION_UBIQUITOUS: Ubiquitous<Station> = {  
  station_id: 'Station ID',  
  station_name: 'Station Name',  
  city: 'City',  
  region: 'Region'  
}
```


지구 반대편을 대응하는 어플리케이션을 위해

실제 비즈니스 관계에 의존하는
유비쿼터스 언어 팩을 사용하여
재사용 가능한 용어집을 코드로 구현하기

```
const CHARGER_UBIQUITOUS: Ubiquitous<Charger> = {  
  station_id: STATION_UBIQUITOUS.station_id,  
  station_name: STATION_UBIQUITOUS.station_name,  
  charger_name: 'Charger Name',  
  charger_id: 'Charger ID'  
  //...  
}
```

지구 반대편을 대응하는 어플리케이션을 위해

실제 비즈니스 관계에 의존하는
유비쿼터스 언어 팩을 사용하여
재사용 가능한 용어집을 코드로 구현하기

```
export const KOR_UBIQUITOUS_LANGUAGE_PACK = {  
  // ...  
}  
  
export const USA_UBIQUITOUS_LANGUAGE_PACK = {  
  // ...  
}  
  
export const JP_UBIQUITOUS_LANGUAGE_PACK = {  
  // ...  
}
```

지구 반대편을 대응하는 어플리케이션을 위해

정말 모든 상황을 대비할 수 있을까요?



우린 답을 찾을 것이다.
—(짜리가 싫다면)—

감사합니다

